Static vs Dynamic Testing

A comprehensive guide to software testing methodologies, model-based testing, and AI-powered quality assurance

Static Testing: Verification Without Execution

Static testing, also known as verification testing, checks software defects without executing code. It's performed early in development to prevent errors and identify issues before runtime.

Key Characteristics

- Performed during white box testing phase
- Reviews code, documentation, and specifications
- Manual and automated assessment
- Cost-effective defect prevention

Static Testing Techniques

Informal Reviews

Team members review documents and provide informal feedback on specifications and requirements.

Walkthroughs

Skilled reviewers or authors explain products to teams while scribes document comments.

Technical Reviews

Peers verify technical specifications and requirements documents for correctness and standards.

Code Reviews

Systematic review of source code without execution, checking syntax, standards, and optimization.

Inspection

Formal review process with strict procedures and checklists to identify and record defects.

50%

Cost Reduction

Defects found earlier are significantly cheaper to fix

Static Testing Benefits

Early Defect Detection

Identifies defects at early stages before execution.
Saves time and effort in later phases

Cost-Effective

Cheaper to detect and fix defects compared to dynamic testing.
Reduces overall development and testing cost

Easy Defect Identification

Detects issues that may not be found during dynamic testing

Improves Development Productivity

Enhances code quality and documentation clarity
Leads to better design and faster development

Identifies Coding Errors

Detects syntax errors and coding issues early
Results in cleaner and maintainable code

•Identifies defects at early stages before execution
•Saves time and effort in later phases
•Identifies defects at early stages before execution
•Saves time and effort in later phases

Limitations of Static Testing

Limited Issue Detection

Cannot identify runtime errors
Some defects appear only during execution

Depends on Reviewer Skills

Effectiveness varies with experience and knowledge. Quality depends heavily on reviewer experience
Human fatigue leads to missed issues

Time-Consuming & Expensive

Requires more time for large and complex projects.

Black +Flake8+Pylint

⚠ Before Linting/Formatting

```
def process_data(data,filter=None,sort_by='date'):
    if filter==None:
       results=[]
       for item in data:
          if item['active']==True:
             results.append(item)
    else:
       results = [i for i in data if i['category']==filter]

    # sort the results
    return sorted(results, key=lambda x: x[sort_by])
```

✅ After Linting/Formatting

```
def process_data(data, filter_=None, sort_by="date"):
    if filter_ is None:
        results = []
        for item in data:
            if item["active"]:
                results.append(item)
    else:
        results = [i for i in data if i["category"] == filter_]

    # Sort the results
    return sorted(results, key=lambda x: x[sort_by])
```

Key Improvements

Naming Conventions

Renamed 'filter' to 'filter_' to avoid shadowing built-in

Consistent Spacing

Proper indentation and spacing around operators

Boolean Simplification

Simplified 'if item['active']==True' to 'if item["active"]'

Consistent Quotes

Standardized on double quotes for strings

Null Comparison

Used 'is None' instead of '==None' for identity check

Variable Naming

Consistent variable naming throughout the function

Popular Static Analysis Tools

Embold

Detects design issues, code smell and architectural problems. Prioritizes issues based on maintainability impact and risk.

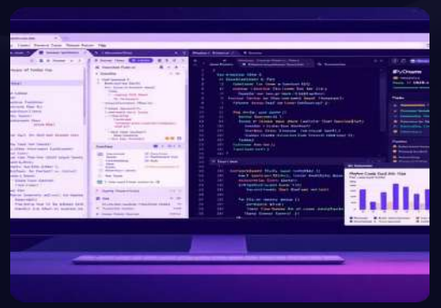
Eg:Consider a Java-based banking application where the same business logic for interest calculation is repeated in multiple classes.

Benefit: Developers know *which problems to fix first* instead of treating all issues equally.

Kiuwan

Focuses on security and compliance with OWASP standards. Integrates with DevOps pipelines for early security detection.

Example: In a web application using Jenkins, when a developer commits code with hardcoded credentials, Kiuwan automatically scans it during the build, detects a security issue (based on OWASP), fails the build, and generates a report with file details and risk level.



PyCharm

Provides real-time static code analysis for Python development. Highlights errors and suggests refactoring while coding.

Example: a Python student record program, PyCharm highlights errors like unused variables, wrong indentation, and incorrect imports, and suggests refactoring—helping fix issues during coding and reducing debugging time.

What is Dynamic Testing?

Dynamic testing analyzes the dynamic behavior of code by executing software and validating outputs against expected outcomes. It confirms software works in conformance with business requirements.

Unlike static testing, dynamic testing requires actual code execution and provides more realistic results through real-time testing.

Execution Required

Tests run with actual input values and validate output behavior

Validation

Compares system response with expected outcomes

Complex Knowledge

Requires deep system understanding from testers

Dynamic Testing Techniques

1

White Box Testing

Also known as clear box testing, it examines internal code workings. Developers test every line of code where test cases derive from source code with known inputs and outputs.

2

Black Box Testing

Tests only Application Under Test functionality. Testers remain unaware of underlying code, verifying expected output generation per requirements.

Whitebox Testing Techniques

1

White Box Testing

Also known as clear box testing, it examines internal code workings. Developers test every line of code where test cases derive from source code with known inputs and outputs.

Statement Coverage

Ensure every single line/statement in the code is executed at least once.

```python
def calculate_discount(price, is_member):
    discount = 0                          # Line 1
    if is_member:                         # Line 2
        discount = price * 0.10           # Line 3
    return price - discount               # Line 4
```

Branch Coverage

Ensure every decision point (True AND False) is tested.

```python
def check_eligibility(age, has_id):
    if age >= 18 and has_id:      # Branch A (True) and Branch B (False)
        return "Eligible"
    else:
        return "Not Eligible"
```

Path Coverage

Test every possible route through the code from entry to exit.

```python
def grade(marks):
    if marks >= 90:           # Path 1
        return "A"
    elif marks >= 75:         # Path 2
        return "B"
    elif marks >= 60:         # Path 3
        return "C"
    else:                     # Path 4
        return "Fail"
```

Black Box Testing Techniques

```
Valid age range: 18 to 60

Boundary test values:
├── 17  → Just below minimum (Invalid)
├── 18  → Minimum boundary  (Valid)
├── 19  → Just above minimum (Valid)
├── 59  → Just below maximum (Valid)
├── 60  → Maximum boundary  (Valid)
└── 61  → Just above maximum (Invalid)
```

Boundary Value Analysis (BVA)

BVA specifically tests the values at and just around boundaries.

2

Black Box Testing

Tests only Application Under Test functionality. Testers remain unaware of underlying code, verifying expected output generation per requirements.

Decision Table Testing

Used when multiple conditions combine to produce different outputs. Creates a table of all condition combinations.

State Transition Testing

Used when the system changes states based on inputs — like a traffic light or ATM.

Equivalence Partitioning

Divide inputs into groups (partitions) where all values in a group should behave the same. Test one value from each group instead of every possible value.

```
Age field – Valid range: 18 to 60

Partitions:
├── Below 18  → Invalid (e.g., test with 10)
├── 18 to 60  → Valid   (e.g., test with 30)
└── Above 60  → Invalid (e.g., test with 70)
```

State Transition Testing

Used when the system changes states based on inputs — like a traffic light or ATM.

Error Guessing Based on tester experience

Guessing where bugs are likely to be.

Benefits & Limitations

Benefits

- Reveals runtime errors, performance bottlenecks, memory leaks
- Verifies module, database, and API integration
- Provides accurate reliability assessment
- Tests real usage scenarios

Limitations

- Time-consuming for complex systems
- Requires significant debugging effort
- Challenging for rare conditions
- May not cover all scenarios

Dynamic Testing Tools

Selenium

Automated testing for web applications

JUnit / TestNG

Unit testing for Java applications

LoadRunner

Performance and load testing

Apache JMeter

Load and stress testing for web apps

Postman

API testing and dynamic validation

Static vs Dynamic Testing

Parameter	Static Testing	Dynamic Testing
Definition	Check defects without executing code	Analyze dynamic code behavior
Objective	Prevent defects	Find and fix defects
Stage	Early development	Later development
Code Execution	Not executed	Fully executed
Cost	Less costly	Highly costly
Time Required	Shorter	Longer
Statement Coverage	100% possible	Less than 50%

Static Code Analysis & Mutation Testing

Static Code Analysis

Static Code Analysis is a technique under **static testing** where the source code is analyzed **without executing it**. It uses automated tools to detect coding errors, security vulnerabilities, and design issues early in development.

Example: Detecting SQL injection by analyzing how user input flows in code.

Mutation Testing

Mutation Testing is a technique under **dynamic testing** where small changes are made to the code and test cases are executed to check whether they can detect those changes.

Purpose: To evaluate the **effectiveness of test cases**.

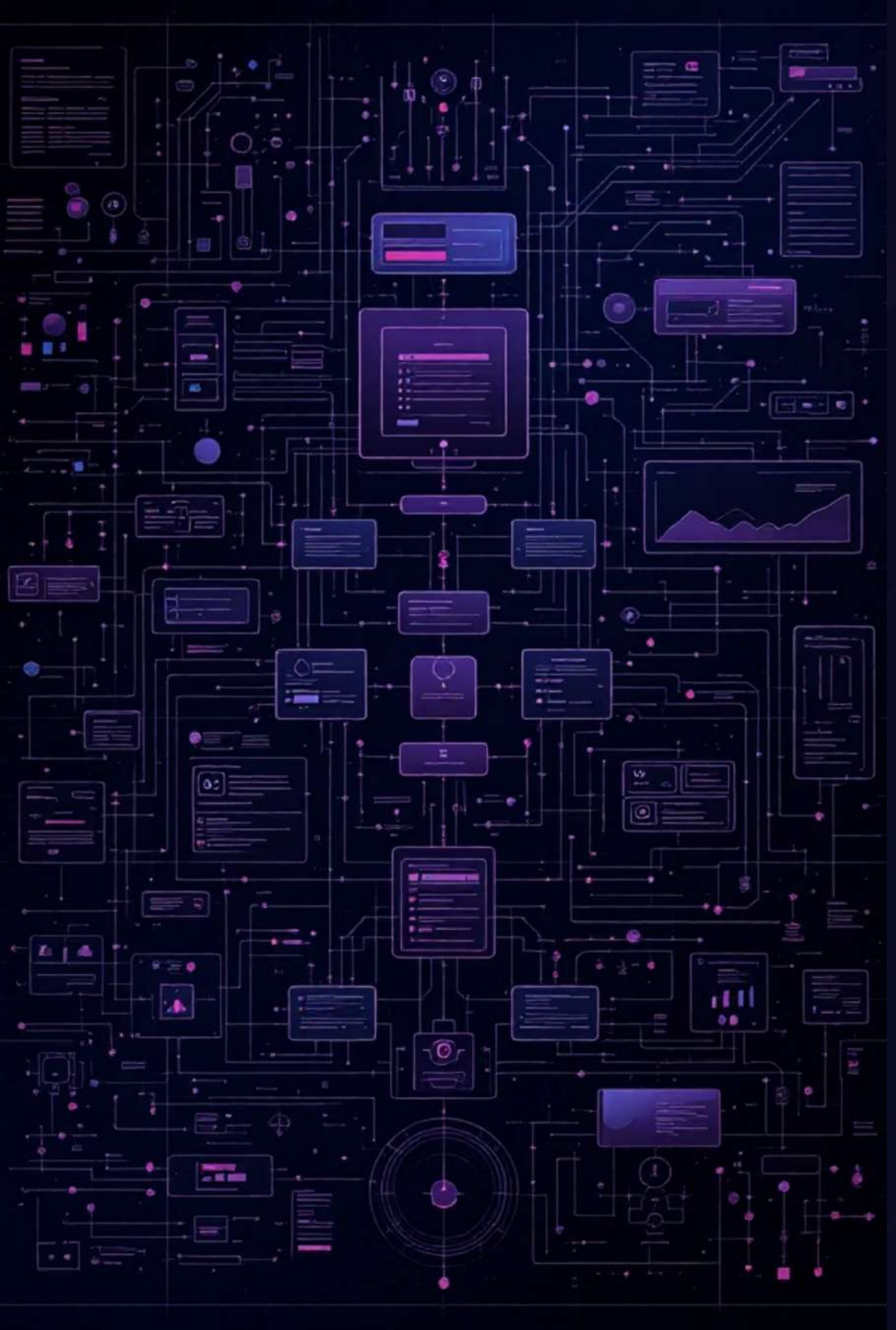
Mutation Score = (Killed Mutants ÷ Total Mutants) × 100

MUTATION SCORE

> 90%Excellent test suite

Common Mutation Operators:

Type	Original	Mutated
Arithmetic	`a + b`	`a - b`
Relational	`x > 0`	`x >= 0`
Logical	`and`	`or`
Assignment	`x = 5`	`x = 0`
Return Value	`return True`	`return False`
Condition	`if x == y`	`if x != y`



Model-Based Testing Overview

Model-based testing generates test cases from models representing system behavior. Test cases derive via online and offline test case models, offering systematic coverage through automated generation.

Model Creation

Design system behavior models

Test Generation

Automatically derive test cases

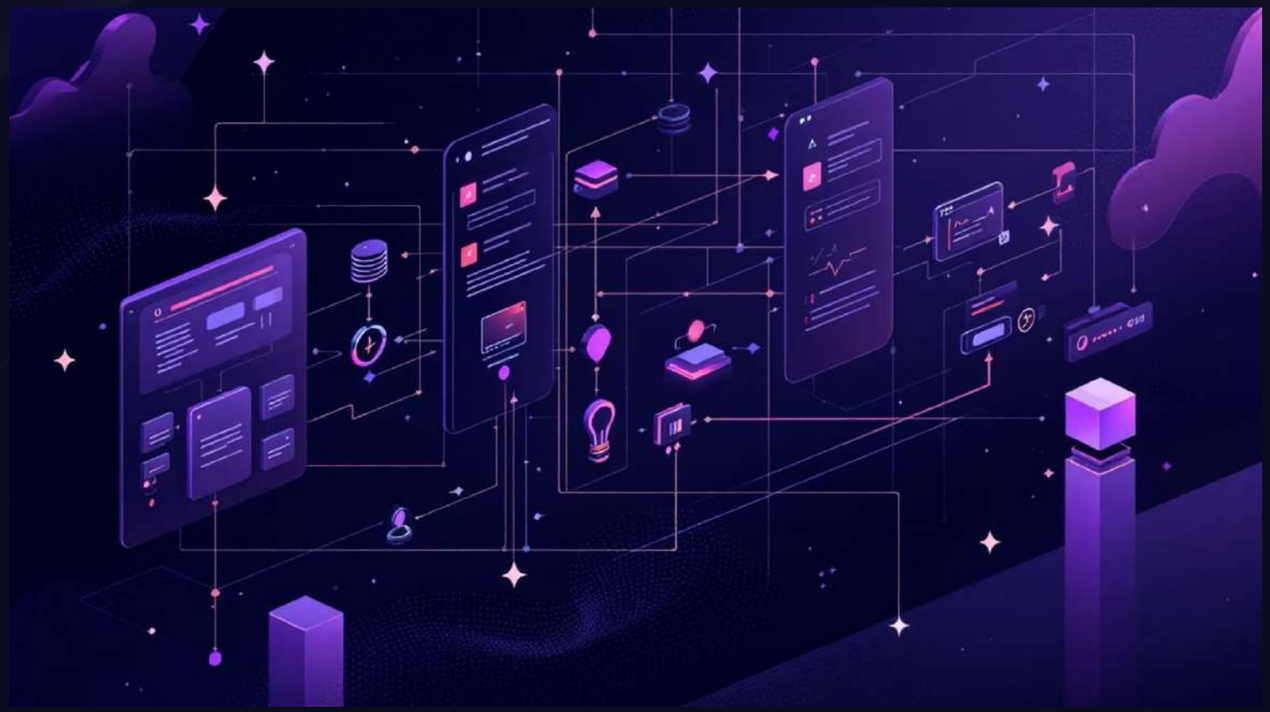
Execution

Run tests against actual system

Model Types & Advantages

Common Model Types

- Statecharts & FSMs
- Markov Models
- Decision Tables
- ERDs & UML diagrams
- Control & Data flow graphs



Early Defect Detection

Identify issues during requirements and design phases

Reduced Maintenance

Update models instead of individual test cases

Comprehensive Coverage

Systematic test generation from complete models

High Automation

Fast test generation and execution

Open-Source MBT Tools

Testsigma

AI-powered test automation with codeless scripting in simple English. Supports web, mobile, API, and desktop testing with 3000+ browser combinations.

fMBT

Multi-platform open-source tool supporting UML diagrams, flowcharts, and state machines. Highly customizable with active community support.

GraphWalker

Graph-based MBT with powerful path generation algorithms. Visual interface simplifies model creation and test visualization for all skill levels.

Commercial MBT Tools

TOSCA

Enterprise-grade MBT with codeless visual test creation using drag-and-drop actions. AI-powered test generation with 90%+ automation rate.

Key Features

- Extensive model support (UML, decision tables)
- Comprehensive reporting and analytics
- Seamless integration capabilities

90%+

Automation Rate

Achieve high test automation with minimal coding

24/7

Support

Dedicated expert assistance and training

3000+

Browser Combos

Cloud execution across extensive environments

AI Testing: The Future of Software Quality

Artificial intelligence is revolutionizing software testing by automating complex tasks, improving efficiency, and enhancing reliability through intelligent test execution and self-healing capabilities.

AI-Powered Testing

AI testing uses artificial intelligence to enhance and streamline testing through automated test execution, data validation, and intelligent error identification.

No-Code Tests

Visual models and drag-and-drop mechanisms create tests without programming knowledge

Self-Healing Tests

Automatically adapts to application changes, reducing manual script maintenance

Smart Prioritization

Machine learning prioritizes high-risk test cases based on recent changes

Visual Validation

Computer vision detects UI regressions with pixel-level accuracy

Types of AI Testing

01

Test Case Generation

AI analyzes user flows and past data to create optimized test cases automatically.

02

Execution Optimization

Prioritizes tests based on risk and code changes to speed up cycles.

03

Self-Healing Automation

Detects and fixes broken locators automatically, reducing maintenance.

04

Test Data Generation

Creates diverse, realistic, context-aware test data for comprehensive testing.

01

Visual Testing

Compares screen layouts across devices to catch UI issues automatically.

02

Flaky Test Management

Identifies unstable tests by analyzing failure patterns and suggests fixes.

03

Predictive Analytics

Analyzes historical defects to predict future failure-prone areas.

04

NLP-Based Testing

Write test cases in plain English, automatically converted to executable scripts.

How to Perform AI Testing

Identify AI-Suitable Areas

Pinpoint repetitive, data-heavy areas where AI adds value like test generation or visual validation.

Collect Historical Data

Use logs, past defects, and user behavior to train AI models for pattern detection.

Automate Test Generation

Use AI tools to auto-generate test cases based on application behavior and code changes.

Apply Intelligent Prioritization

Machine learning analyzes risk to prioritize tests, saving time and increasing defect detection.

Enable Self-Healing

AI-powered scripts automatically update when UI elements change, minimizing maintenance.

Use Visual AI

Employ computer vision for pixel-level UI checks and visual regression detection.

Monitor and Learn

Integrate AI into CI/CD pipeline to continuously learn from results and improve predictions.

Two Implementation Approaches

Building Custom AI (From Scratch)

Use Cases: Auto-generate tests from user flows, detect dynamic UI changes, create domain-specific NLP-based authoring.

Benefits: High customization, seamless integration, complete control over data and privacy.

Challenges: Requires AI/ML expertise, higher costs, longer timelines, continuous training needed.

Leveraging Proprietary Tools

Benefits: Fast implementation, no AI expertise needed, reduced maintenance, easy CI/CD integration, suitable for Agile teams.

Challenges: Licensing costs, limited customization compared to custom solutions.

AI Testing vs Manual Testing

Test Creation

AI: Auto-generates from user flows and code analysis

Manual: Tester writes test cases manually

Execution

AI: Smart execution with prioritization and self-healing

Manual: Step-by-step manual execution by tester

Maintenance

AI: Automatically fixes UI changes (self-healing locators)

Manual: Needs frequent manual updates if UI changes

Data Generation

AI: Generates realistic and diverse test data automatically

Manual: Tester prepares test data manually

Speed & Efficiency

AI: Faster execution with reduced human effort

Manual: Slower execution, highly time-consuming

Accuracy

AI: Reduced human errors, detects patterns and edge cases

Manual: Prone to human errors or oversight

Top AI Testing Tools

BrowserStack Low Code

Creates tests without code, combines test recorder with AI-powered self-healing and real-device cloud testing.

EvoSuite

Uses search-based AI and evolutionary algorithms to automatically generate unit tests for Java applications.

EggPlant

Leverages AI-driven model-based testing to simulate real user behavior and validate complex workflows.

TestCraft

Creates, manages, and maintains Selenium-based tests with self-healing capabilities and reduced maintenance.

Watir

Ruby-based automation framework supporting AI-enhanced testing with intelligent generation and analysis.

Parasoft

Integrates AI to optimize test coverage, identify risk areas, and automate quality assurance across APIs.

Additional tools include Test.ai, Aqua ALM, Sealights, Vize.ai, TestCafe Studio, ZapTest, Healium, and Sofy.ai.

Why BrowserStack for AI Testing

Test Recorder

Captures user actions and converts them into automated tests with functional and visual validations.

Visual Validation

Adds checkpoints to confirm UI display and identify visual regressions automatically.

AI-Powered Self-Healing

Detects UI changes and automatically updates test steps to prevent failures.

Low-Code Agent

Converts natural language prompts into executable test steps without coding.

Cross-Browser Testing

Runs tests on thousands of real desktop browsers and mobile devices in the cloud.

CI/CD Integration

Runs tests on schedule or triggers via REST APIs for continuous validation.

Best Practices for AI Testing

Understand Capabilities

Thoroughly understand AI tool capabilities and limitations. Align tools with specific testing objectives and recognize their constraints.

Clear Strategy

Define clear testing strategy with objectives, test cases, metrics, and expected outcomes aligned with quality assurance goals.

Real-World Scenarios

Validate AI tool performance under real-world conditions, including unexpected or edge cases using diverse datasets.

Monitor Performance

Continuously monitor AI tool performance and audit results. Implement regular updates and retrain models as needed.

Test for Bias

Regularly test for bias in AI-powered tools, ensuring fairness metrics and diverse datasets minimize potential bias.

Explainable AI

Opt for AI tools with explainable AI (XAI) features allowing testers to understand decision-making processes.

Best Practices (Continued)

1

CI/CD Integration

Integrate AI testing tools into CI/CD pipeline for seamless automated testing and faster feedback loops.

2

Human Oversight

Include human oversight, especially for high-stakes applications. Human intervention crucial for complex scenarios and ethical standards.

3

Scalability Testing

Test scalability to handle large datasets and high volumes of test cases without performance degradation.

4

Predictive Testing

Use AI for predictive testing, analyzing trends to suggest likely failure areas and prioritize critical coverage.

5

Balance Automation

Maintain balance between automated AI testing and manual testing. Use AI for repetitive tasks, manual for user-experience and exploratory tests.

Key Takeaways

Complementary Approaches

Static testing prevents defects early while dynamic testing validates runtime behavior. Both are essential for comprehensive quality assurance.

Model-Based Efficiency

MBT reduces maintenance costs through automatic test generation from models, ensuring comprehensive coverage across all system behaviors.

AI Revolution

AI-powered testing delivers faster creation, self-healing capabilities, and predictive analytics, reducing costs by 25-75% while improving coverage.